MC809 Computação Distribuída
Laboratório 6

Nome: Arthur Costa Barreto
RA: 206325
Data: Outubro de 2026

1. Introdução
1.1. Descrição da Proposta do Laboratório

Neste laboratório, exploramos um framework de aprendizado federado chamado *Flower*. Mais especificamente, iremos realizar simulações de treinamentos de modelos federados utilizando o método FedAvg como agregador e distintas características de distribuição dos dados e da quantidade de clientes utilizados em cada rodada de treino.

Diferente dos exemplos que vimos ao longo da explicação do experimento, que tratava do treinamento de modelos cross-silo, iremos explorar o treinamento cross-device. Esse método de treinamento envolve separar os dados para que existam clientes com dados de avaliação e clientes com dados de treino, ou seja, clientes com dado de treino nunca serão utilizados para avaliar o modelo e, em minha implementação, vice-versa.

1.2. Divisão dos arquivos

Para este exercício, existem 5 arquivos principais:

- client.py: Arquivo que contém a definição da classe Cliente;
- server.py: Arquivo que contém a definição da classe Servidor;
- simulation.py: Arquivo que contém a definição da classe Simulation e é o executável para rodar experimentos individuais;
- run_experiments.sh: Script contendo as configurações de round, fraction-fit e alpha desejados para gerar diferentes resultados experimentais.
- plots.ipynb: Notebook apenas de visualização dos resultados de treino e avaliação para cada configuração do script;

2. Mudanças nos arquivos originais
2.1 Classe do cliente

Para poder identificar quais clientes serão utilizados para treino e quais serão utilizados para avaliação, agora passam-se os valores de *num_train_clients* e define-se um cliente como de treino se o cid dele for menor do que o valor de *num_train_clients* passado na inicialização da classe.

Complementando essa abordagem, para o servidor identificar com qual dos tipos de cliente está se comunicando, criou-se um método chamado *get_properties* que retorna justamente *is_train_client* e o cid do cliente. Por fim, o método *load_data* agora utiliza o resultado de *is_train_client* para definir que os clientes de treino dividem o split train em *num_train_clients* partições e os de avaliação dividem o split test entre os demais clientes.

2.2 Classe do servidor

Primeiramente foi criada uma classe adicional de suporte ao servidor, chamada de *GrupoCriterion*, subclasse de *flwr.server.criterion.Criterion*. Ela possui apenas um método, *select*, que possui a finalidade de decidir quais clientes serão utilizados para treino e avaliação; e é inicializada com *cid_map*, utilizado para mapear o id real do nó para o id do cliente, *num_train_clients* e *is_train*. O método *select* recebe um objeto cliente como parâmetro e busca o seu cid no *cid_map* para definir se o cliente é de treino ou avaliação.

A classe Servidor agora passa a ser inicializada também com *num_train_clients* e *num_eval_clients* (*num_clients* - *num_train_clients*), além de um cid_map (dicionário) vazio. Esse *cid_map* é preenchido utilizando o *client_manager* durante a execução do código. A partir do *client_manager* buscam-se todos os itens (clientes disponíveis) e utiliza-se o *get_properties* definido na classe do cliente para obter o cid. Com o cid em mãos, o cid_map é preenchido com a chave sendo o valor do nó e o valor sendo o cid obtido do *get_properties*. Isso é necessário, pois o *ClientProxy.cid* visto pelo servidor é um *node_id* aleatório, diferente do cid recebido pelo client_fn.

Por último, alterou-se também a função *configure_fit* para que o valor de *num_clients* passado ao *client_manager* não seja o *fraction_fit* sobre todos os clientes disponíveis, mas apenas o *fraction_fit* sobre os clientes definidos como de treino. O mesmo raciocínio é utilizado para passar o valor de *num_clients* para *configure_evaluate*, mas agora apenas com a porção de clientes de avaliação.

2.3 Classe de simulação

A classe de simulação permanece praticamente igual, com a única diferença sendo que passamos o *num_train_clients* quando inicializamos a classe Servidor e passamos um diretório para salvar os resultados de treino e avaliação (results/<experimento>), a fim de melhor separar os resultados de cada experimento rodado e detalhado na seção a seguir. Também limitou-se as execuções à CPU, por conta da incompatibilidade entre a versão do *TensorFlow* exigida pelo *Flower 1.9* e do hardware disponível.

3. Metodologia

Para verificar a influência de dados não iid e do número de clientes selecionados para cada rodada de treino, fixou-se o número de rodadas de treino em 20 e variou-se o valor de alpha de Dirichlet e de fraction_fit (fração do total de clientes selecionados para treino utilizados em cada rodada). Além disso, o experimento foi feito com um total de 10 clientes, sendo 8 de treino e 2 de avaliação.

Mais especificamente, os seguintes valores foram utilizados para combinações experimentais:

1. alpha de Dirichlet $\in$ {0,1; 1; 100}
2. fraction_fit $\in$ {0,5; 1}
3. treino com dados iid (sem alpha associado, pois Dirichlet é utilizado para distribuições não iid)

O número de clientes selecionados é baixo para um cenário cross-device real, mas os resultados mostrados a seguir nos trazem informações relevantes sobre o tema explorado no

laboratório. Como só temos 2 clientes de avaliação, a média e o intervalo de confiança dos resultados podem ficar ruidosos, especialmente quando diminuímos o valor de alpha.

4. Resultados

A seguir, veja os resultados obtidos com cada configuração de treino nos plots de acurácia e perda dos modelos:

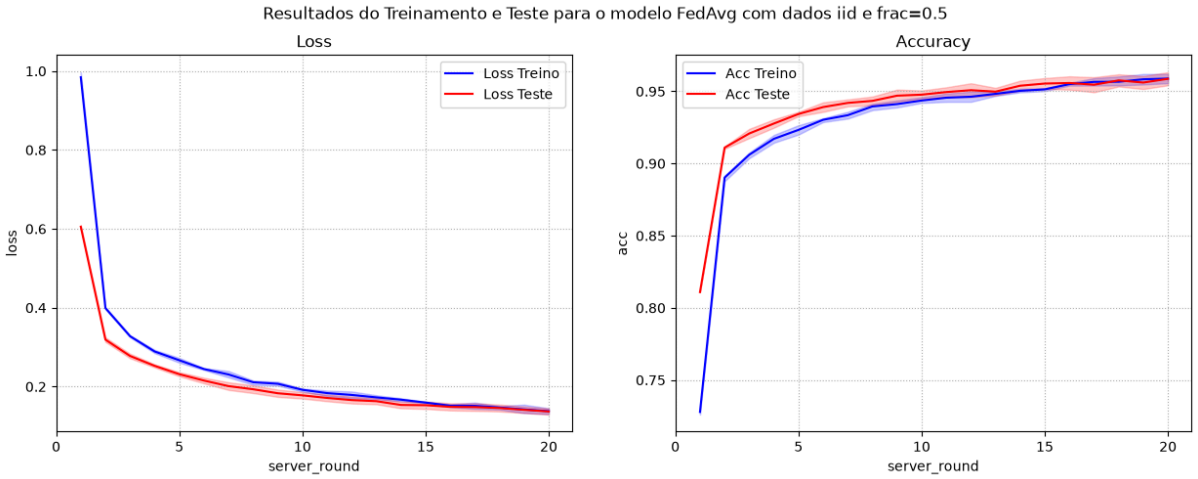


Figura 1: Acurácia e loss para dados iid e fraction_fit=0.5

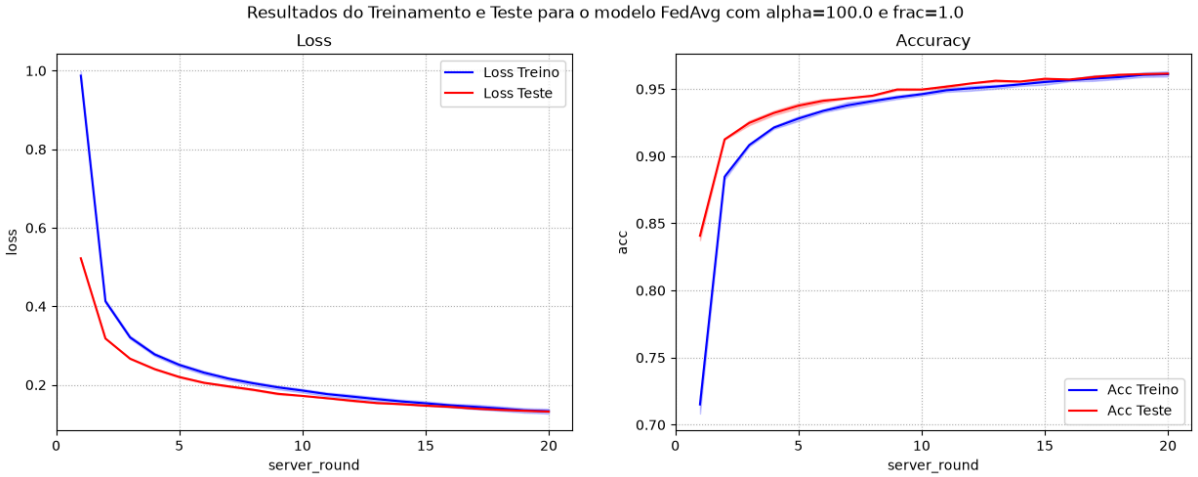


Figura 2: Acurácia e loss para alpha=100 e fraction_fit=1.0

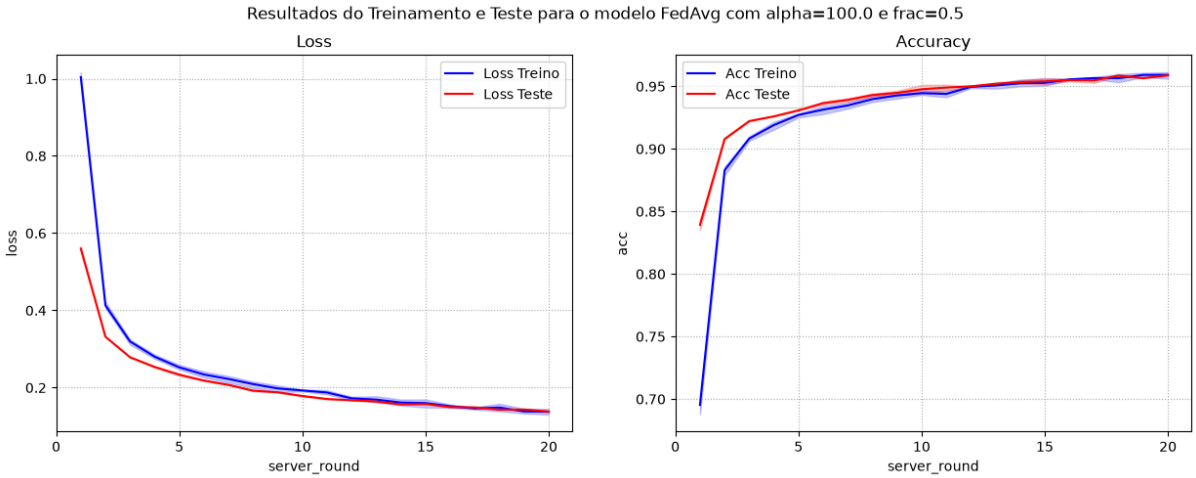


Figura 3: Acurácia e loss para alpha=100 e fraction_fit=0.5

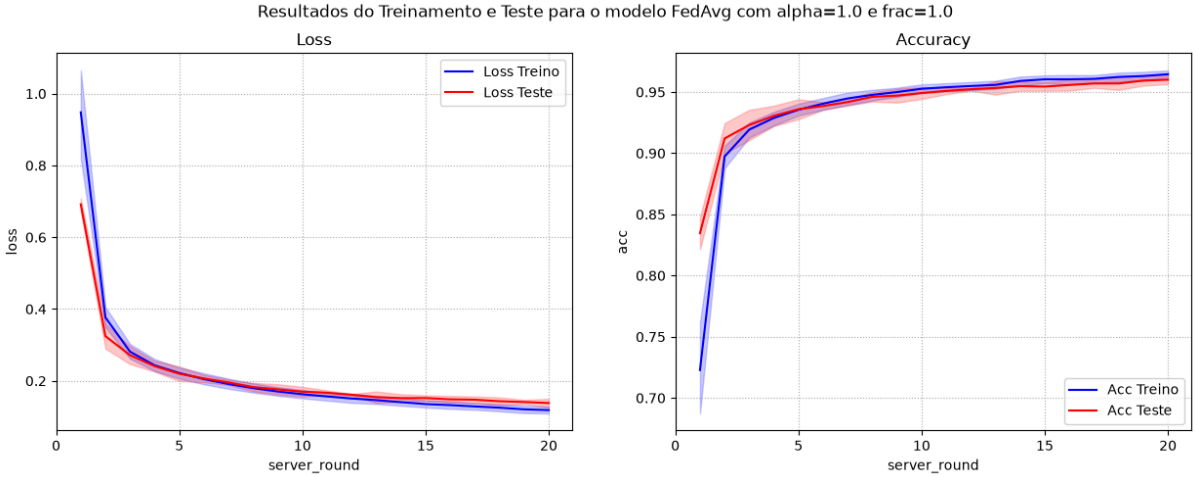


Figura 4: Acurácia e loss para alpha=1 e fraction_fit=1.0

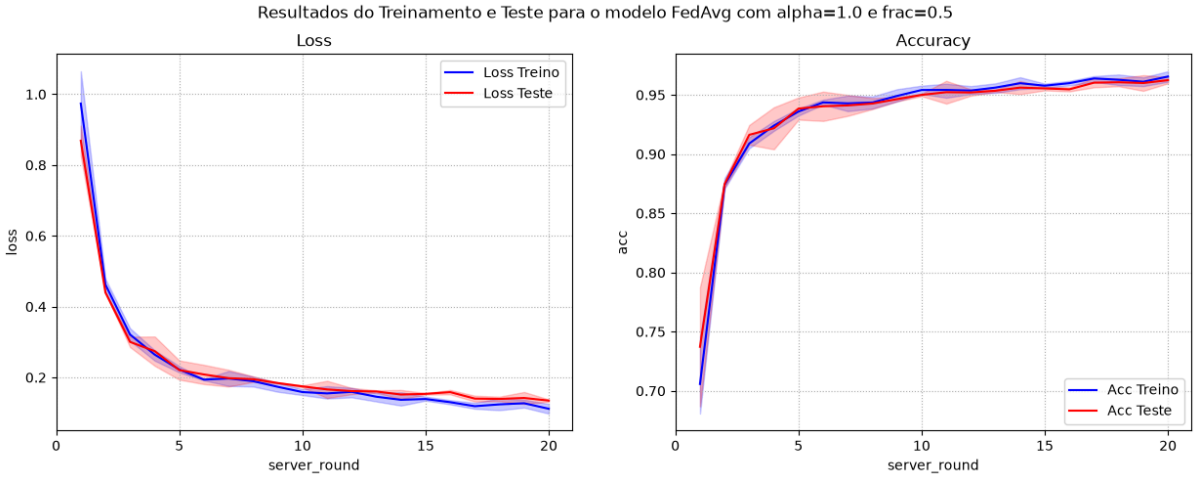


Figura 5: Acurácia e loss para alpha=1 e fraction_fit=0.5

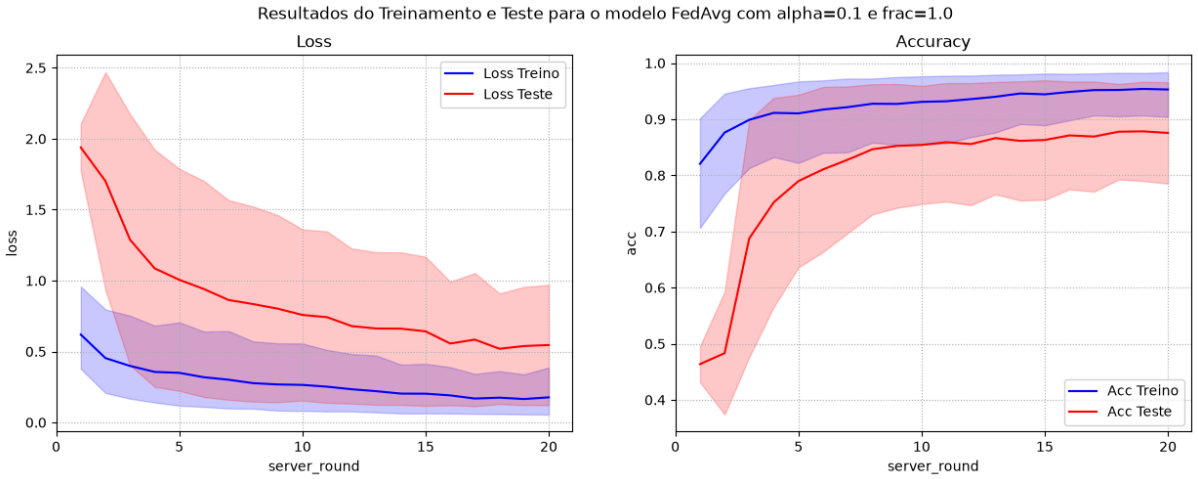


Figura 6: Acurácia e loss para alpha=0.1 e fraction_fit=1.0

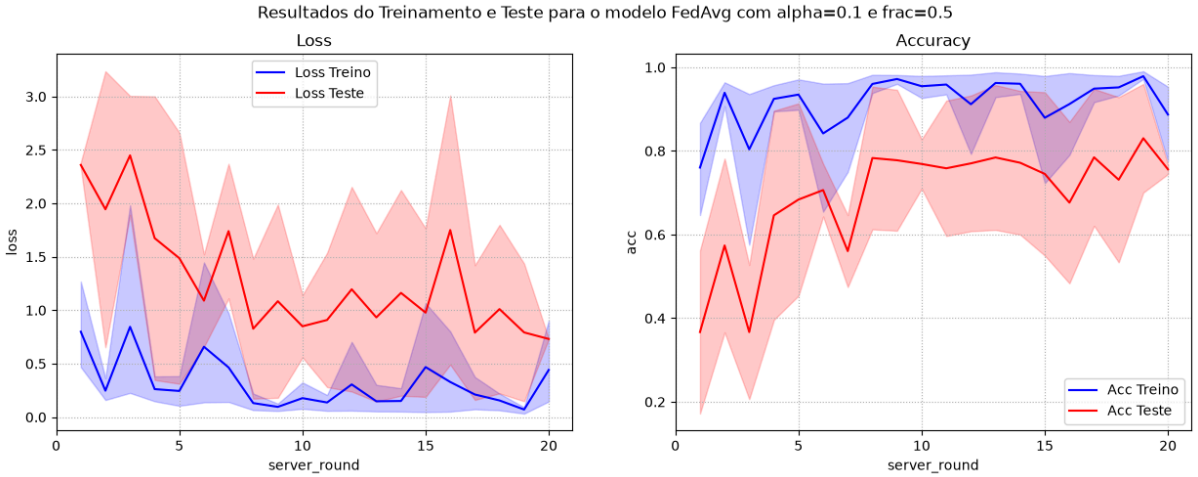


Figura 7: Acurácia e loss para alpha=0.1 e fraction_fit=0.5

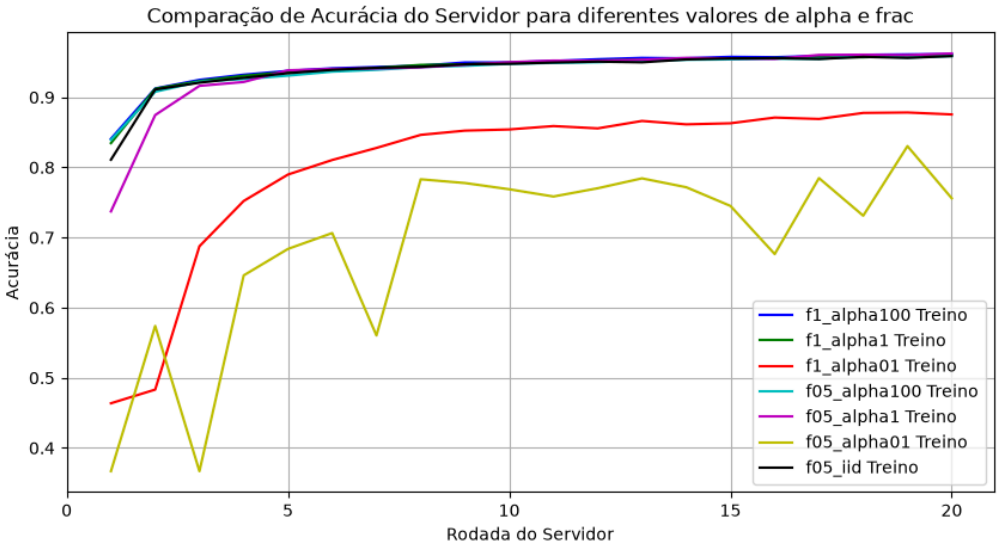


Figura 8: Comparação do resultado do modelo global entre experimentos

5. Conclusões

A partir dos resultados obtidos, é possível concluir que:

1. **Diminuir o alpha tem como consequência maior dispersão da acurácia do modelo global entre os clientes**.

Isso fica claro pelo aumento da área sombreada conforme o valor de alpha diminui. Note que nas figuras 1, 2 e 3 essa área é praticamente nula, mas nas figuras 6 e 7 ela é a maior dentre todas do experimento.

A causa disso é que os modelos estão recebendo pesos globais treinados em todas as classes, mas suas distribuições de dados contêm classes específicas. Dessa forma, em uma rodada o modelo global pode estar bom para classificar uma porção de classes, mas se um cliente possui dados de classes que fogem dessa porção, o modelo se perde e os resultados de acurácia e perda não são bons. Por outro lado, o contrário também é válido, se o cliente possui justamente as classes que o modelo se especializou, seu resultado será acima da média.

2. **Diminuir o fraction_fit faz os resultados de treino e avaliação ficarem menos consistentes entre rodadas**.

Isso ocorre porque clientes diferentes têm suas próprias distribuições de classes, ou seja, isso potencializa o problema do ponto anterior. Por exemplo, se em uma rodada os clientes selecionados possuírem dados apenas de 5 classes e na outra rodada possuírem apenas das outras 5 classes, haverá um gap gigantesco entre os resultados até que possa convergir, se convergir.

Esse ponto fica claro quando as figuras 6 e 7 são analisadas conjuntamente. Note que o resultado entre rodadas consecutivas da figura 7 mostra picos acentuados quando comparado com os da figura 6, que não mostra picos, mas uma tendência de convergência.

No entanto, para alpha maior ou igual a 1, esse efeito é praticamente desprezível. Ele só se torna relevante quando os dados estão muito não iid, ou seja, quando o alpha é baixo.

3. **No geral, o modelo global tende a convergir após certa quantidade de rodadas**

A partir da figura 8, vemos que o modelo global vindo do servidor converge para o mesmo valor em todos os experimentos, exceto para os casos extremos de alpha=0.1. Isso mostra que, mesmo com classes não identicamente distribuídas, o modelo global treinado de forma federada consegue aprender e pontuar bem em dados não vistos durante o treinamento. No entanto, se a distribuição de classes for muito concentrada nos clientes (alpha muito baixo), o modelo perde boa parte de sua capacidade – acurácia cai de ~95% para ~76% no pior caso – mesmo que a acurácia global mostre uma tendência de evolução entre rodadas.